Symbolic software

Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.4: End-to-End Encrypted Cloud Storage

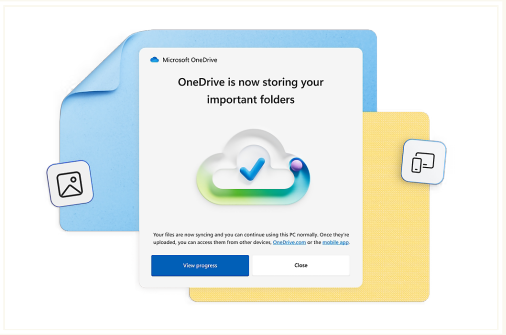
Nadim Kobeissi
https://appliedcryptography.page

Section 1

Introduction

Cloud storage today

- Cloud storage is a prevalent computing paradigm today
- Outsources data storage to third-party services
- Benefits for users:
  - No need to worry about backups
  - High data availability
  - Access data from anywhere
- Major providers (Google, Microsoft, Apple, Amazon) control encryption keys



Microsoft OneDrive is an example of a popular cloud storage service.

Regular vs End-to-End Encrypted Cloud Storage

Regular Cloud Storage
(e.g., OneDrive, Google Drive)

- **Transport encryption:** TLS protects data in transit
- **Encryption at rest:** Provider encrypts on their servers
- **Provider controls keys:** Can decrypt your data

E2E Encrypted Storage
(e.g., MEGA, Nextcloud)

- **Client-side encryption:** Data encrypted before upload
- **User controls keys:** Provider never sees plaintext
- **Confidentiality:** Provider learns nothing about content

Regular vs End-to-End Encrypted Cloud Storage

Regular Cloud Storage
(e.g., OneDrive, Google Drive)

Risks:

- Provider breach exposes data
- Government requests
- Insider threats
- Provider can analyze content

E2E Encrypted Storage
(e.g., MEGA, Nextcloud)

Guarantees:

- Provider breach reveals nothing
- Government gets encrypted data
- No insider access to content
- Provider can only analyze metadata

End-to-End Encryption (E2EE) for cloud storage

- Alternative approach: Users encrypt before uploading
- Users control their own encryption keys
- Security guarantees:
  - Cryptographic confidentiality
  - Authenticity and integrity protection
  - Provider can't read file contents
  - Provider can't modify files undetectably
- Security maintained even if service is compromised
- Ideally secure against malicious service providers

Key management challenges

- Key management delegated to users
- Humans are poor at memorizing cryptographic keys
- Security typically bootstrapped from passwords
- Fundamental limitation: Dictionary attacks possible
- Trade-off between convenience and security
- Strong password policies can partially mitigate risks

Password hashing

- Users typically access E2EE storage via passwords
- Passwords must be converted to cryptographic keys
- Common approach: password hashing[a]
- Popular password hashing functions:
  - **PBKDF2**: Widely deployed but aging
  - **scrypt**: Memory-hard against ASICs
  - **Argon2**: Winner of Password Hashing Competition
- KDFs slow down brute-force attacks
- Still vulnerable to dictionary attacks with weak passwords

[a] This was previously covered in our *Collision-Resistant Hash Functions* topic.

The authentication vs encryption dilemma

- Traditional services: Server stores password hash
- E2EE requirement: Server must never see encryption key
- Three common approaches:
  - **Dual-password:** Separate login and encryption passwords
  - **SRP (Secure Remote Password):** Password-authenticated key exchange
  - **Hardware-backed authentication:** Passkeys, Apple Secure Enclave, etc.
- Trade-offs:
  - Dual-password: Poor usability, users often reuse
  - SRP: Complex implementation
  - Hardware-backed: Requires specific devices/platforms
- MEGA uses custom authentication with encryption keys

The mud puddle test

- **What is the mud puddle test?**
  - Thought experiment for E2EE systems
  - “If you drop your device in a mud puddle and it’s destroyed…”
  - “And then slip in that mud puddle and suffer from password amnesia…”
  - “…can you recover your data on a new device?”
- **In true E2EE:**
  - Lost password = lost data forever
  - No backdoor for the provider
  - No “forgot password” that maintains E2EE
- **The fundamental trade-off:**
  - Security vs recoverability
  - Provider can’t help without breaking E2EE

Password recovery expectations vs reality

- **User expectations:**
  - “Forgot password” should always work
  - Data should never be permanently lost
  - Trained by non-E2EE services (Gmail, Facebook)
- **Common mitigation strategies:**
  - **Recovery keys:** Long random string users must store
  - **Social recovery:** Threshold sharing among contacts
  - **Security questions:** Weaker but familiar to users
- **Each approach has trade-offs:**
  - Recovery keys: Often lost or not backed up
  - Social recovery: Complex setup, trust issues
  - Security questions: Weak against targeted attacks

Apple's Advanced Data Protection

- Apple introduced optional E2EE for iCloud in 2023[a]
- Called "Advanced Data Protection" (ADP)
- Covers most iCloud data categories:
  - Photos, Notes, iCloud Backup, Messages backup
  - iCloud Drive, Reminders, Safari bookmarks
- Apple's approach to recovery:
  - **Option 1:** User-managed recovery key
  - **Option 2:** Apple escrow (default)
  - Users must explicitly opt-in to ADP
- Fundamental tension: Security vs recoverability
- Apple chose user choice over mandatory E2EE

[a] https://support.apple.com/en-us/102651

Multi-device key management

- Users expect seamless multi-device access
- Each device needs access to encryption keys
- Key synchronization approaches:
  - **Password-based:** Derive keys on each device
  - **Device pairing:** Transfer keys device-to-device
  - **Cloud keychain:** Encrypted key storage (circular problem)
- Additional challenges:
  - Device revocation when lost/stolen
  - Key rotation after compromise
  - Offline device synchronization

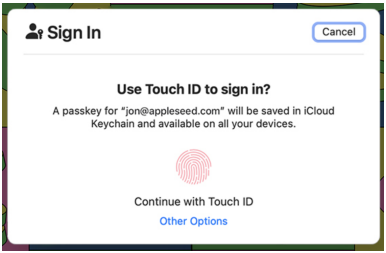
Research directions in usable key management

- **Threshold cryptography:** Split keys across multiple locations
- **Hardware tokens:** FIDO2/WebAuthn/Passkeys for encryption key storage
- **Biometric key derivation:** Fuzzy extractors from fingerprints
- **Social key recovery:** Friends/family as key custodians
- Open problems:
  - Balancing security with user-friendliness
  - Recovery without trusted third parties
  - Protecting against coercion/rubber-hose attacks
- Most solutions add complexity or trust assumptions

Passkeys: A new authentication paradigm

Quick aside

- **Passkeys** are a password replacement based on WebAuthn/FIDO2
- Built on public-key cryptography:
  - Private key stored securely on device
  - Public key registered with service
  - Authentication via cryptographic signatures
- Key advantages over passwords:
  - Phishing-resistant by design
  - No shared secrets with server
  - Automatic strong credentials
- Apple: iCloud Keychain, Microsoft: Windows Hello…



Passkey dialog on macOS.

WebAuthn PRF: symmetric keys from passkeys

Quick aside

- WebAuthn's **PRF (Pseudo-Random Function)** extension enables key derivation
- Key derivation process:
  - PRF can be evaluated during credential creation/assertion
  - Produces 32-byte outputs suitable for encryption keys
  - Supports two inputs per operation (for key rotation)
- Implementation typically uses HMAC-SHA256 internally

Using passkeys for key management

Quick aside

- Potential architecture combining passkeys with E2EE:
  - User authenticates with passkey (no password needed)
  - PRF extension derives encryption keys on-demand
  - Keys never leave the authenticator/device
- Benefits:
  - No password-derived keys (stronger security)
  - Hardware-backed key storage
  - Seamless multi-device via platform sync
- Current limitations:
  - PRF extension not universally supported
  - Security keys may not support PRF evaluation at creation
- Represents potential future direction for usable E2EE key management and authentication more generally

File sharing challenges

- Primary feature: Sharing files with selected users
- Recipients need decryption keys
- Keys cannot be sent unprotected via untrusted provider
- Common solutions:
  - Encrypt file key under recipient's public key
  - Use out-of-band mechanisms

E2EE cloud storage providers

- Major providers:
  - MEGA: 300M users, 150B files, 1000 PB data
  - Nextcloud
  - Apple iCloud (optional E2EE since 2023)
- Smaller providers:
  - Icedrive, pCloud, Seafile, Tresorit
  - Tens of millions of users collectively

Security concerns with current E2EE providers

- Recent analyses found attacks on MEGA and Nextcloud
- Both providers use complex, ad-hoc cryptographic designs
- Lack formal security guarantees
- MEGA's challenges:
  - Migration infeasible - requires user participation
  - Initial minimal patches proved insufficient
  - Required second round of complex patching
- Nextcloud's response:
  - Forced to disable E2EE file sharing entirely
  - Significant feature regression
  - Required full redesign

Current state of E2EE Cloud Storage

- Most providers rely on proprietary, unanalyzed designs
- Highlights need for formally analyzed cryptographic designs
- Security confidence lags behind E2EE messaging and browsing
- Less design input from cryptographic research community
- Need for standardized, proven security approaches
- **In this topic, we'll cover:**
  1. What went wrong with MEGA
  2. What went wrong with Nextcloud
  3. A quick look at WhatsApp encrypted cloud backups
  4. An attempt by cryptographers to design a provably secure E2EE Cloud Storage protocol

Section 2

Attacks on MEGA and Nextcloud

Section 2

Attacks on MEGA and Nextcloud

Subsection 2.1

Attacks on MEGA

MEGA: Overview

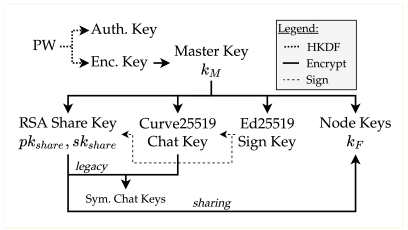
- Founded in 2013 by Kim Dotcom (post-Megaupload)
- One of the largest E2EE cloud storage providers
- **Key statistics**:
  - Over 300 million registered users[a]
  - 150+ billion files stored
- **Core features**:
  - Client-side encryption before upload
  - Secure file sharing via encrypted links
  - Cross-platform support (web, desktop, mobile)
  - Free tier with 20 GB storage
- All encryption performed in JavaScript (web client)

MEGA's logo.

[a] https://blog.mega.io/mega-reaches-300-million-users-milestone

MEGA: Key hierarchy

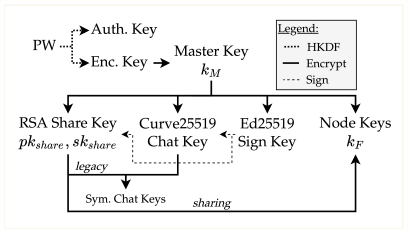
- Password serves as root of trust for entire key hierarchy
- Key derivation from password:
  - **Authentication key:** Identifies user to MEGA
  - **Encryption key:** Encrypts the master key
- **Master key:** Randomly generated, encrypts all other keys



MEGA's key hierarchy. The master key encrypts the share, chat, sign and node keys using AES-ECB. Source: https://appliedcryptography.page/paper/#mega-awry

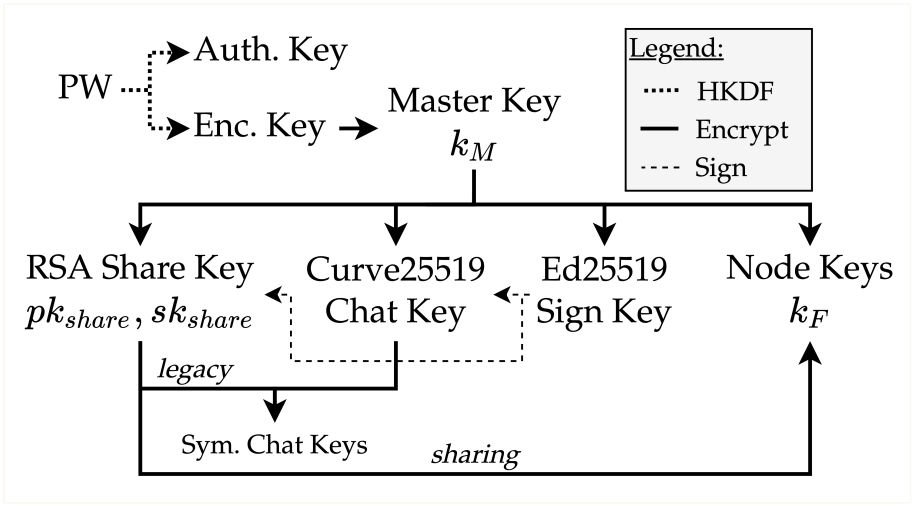
MEGA: Key hierarchy

- Asymmetric key pairs for each account:
  - **RSA key pair:** For data sharing
  - **Curve25519 key pair:** For chat key exchange
  - **Ed25519 key pair:** For signing other keys
- **Per-node keys:** New key for each file/folder
- All keys encrypted with master key using AES-ECB
- Encrypted keys stored on MEGA servers
- Enables multi-device access with password only



MEGA's key hierarchy. The master key encrypts the share, chat, sign and node keys using AES-ECB. Source: https://appliedcryptography.page/paper/#mega-awry

MEGA: Where's the attack?



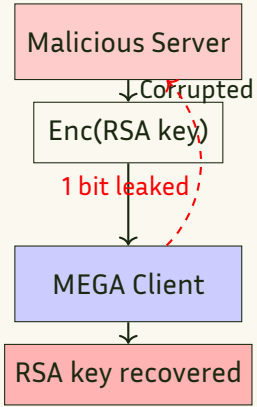
MEGA's key hierarchy. The master key encrypts the share, chat, sign and node keys using AES-ECB. Source: https://appliedcryptography.page/paper/#mega-awry

MEGA: Key hierarchy

- Password-to-key derivation
- MEGA uses HKDF for deriving keys from password
- **Problem:** HKDF is not a password hash!
  - HKDF designed for key derivation from high-entropy inputs
  - No computational hardness against brute-force
  - Passwords have low entropy
- **Should use:** Argon2, scrypt, or PBKDF2
  - Memory-hard (Argon2, scrypt) or iterative (PBKDF2) functions
  - Configurable iteration counts
  - Resistance to GPU/ASIC attacks (especially memory-hard functions)
- Enables offline dictionary attacks on user passwords

Attack 1: RSA key recovery attack

- **Target:** User's RSA private key (for sharing)
- **Exploit:** No integrity protection on encrypted RSA key
- **Attack vector:**
  - Malicious server modifies encrypted RSA key
  - Client decrypts corrupted key
  - RSA-CRT implementation leaks information
- **Oracle construction:**
  - Each login attempt leaks 1 bit
  - About RSA modulus factors



512 logins

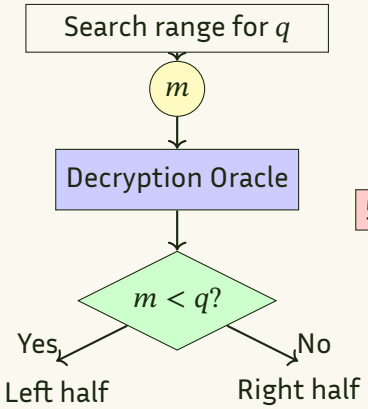
Attack 1: High-level overview

- **Threat model:** Malicious service provider controls MEGA infrastructure
- **Attack strategy:**
  1. Modify encrypted RSA private key (no integrity protection)
  2. Use modified key to create decryption oracle
  3. Binary search to recover RSA prime factor
- **Key insight:** RSA-CRT implementation leaks information
  - Modified key causes different behavior for $m < q$ vs $m \geq q$
  - Where q is one of the RSA primes
  - Observable through session ID responses
- **Attack efficiency:**
  - Binary search requires ~512 login attempts[a]
  - Each login leaks 1 bit of information about q

[a] Later improved to 2 logins: https://appliedcryptography.page/paper/#mega-recovery

Attack 1: The decryption oracle

- **Setup:**
  - Server sends corrupted RSA key
  - Key has modified $u' \neq u = q^{-1} \bmod p$
- **Oracle mechanism:**
  - Server chooses message m
  - Sends encrypted $[m]_{pk}$ as session ID
  - Client decrypts with corrupted key
- **Observable behavior:**
  - If $m < q$: Returns $\text{sid}' = 0$
  - If $m \geq q$: Returns $\text{sid}' \neq 0$



512 iterations

Quick reminder: What does x^{-1} mean?

In modular arithmetic:

- x^{-1} is the **modular inverse** of x
- It's the number that, when multiplied by x, gives 1
- $x \cdot x^{-1} \equiv 1 \pmod{n}$

Example:

- If $x = 3$ and $n = 7$
- Then $x^{-1} = 5$ because $3 \cdot 5 = 15 \equiv 1 \pmod{7}$

In group elements:

- If $A = g^a$, then $A^{-1} = g^{-a}$
- $A \cdot A^{-1} = g^a \cdot g^{-a} = g^{a-a} = g^0 = 1$
- It's the element that "cancels out" A

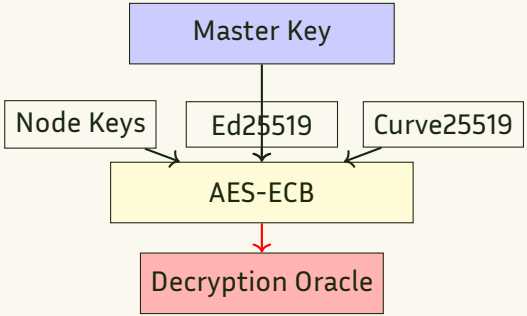
Key property

$(A^c)^{-1} = A^{-c}$

This is why we can rewrite expressions!

Attack 2: Plaintext recovery attack

- **Goal:** Decrypt any AES-ECB ciphertext under master key
- **Impact:**
  - All node keys (file/folder encryption)
  - Ed25519 signing key
  - Curve25519 chat key
  - Complete confidentiality breach
- **Attack requirements:**
  - RSA key from Attack 1
  - Ability to modify key ciphertexts
  - Control authentication plaintexts



All keys recovered!

MEGA's patches: A cautionary tale

- **MEGA's response to attacks:** Added sanity checks instead of proper fixes
- **New paper (2023):** "Caveat Implementor! Key Recovery Attacks on MEGA"[a]
- **Key findings:**
  - MEGA's patches introduced *new* vulnerabilities
  - Detailed error reporting became an attack vector
- **The irony:** Ad-hoc fixes to ad-hoc designs created more problems
  - ECB encryption oracle in MEGAdrop feature
  - Distinguishable error messages leak information
  - Patches made attacks *easier*, not harder
- **Contrast with Nextcloud:** Had the wisdom to pause and redesign
- **Lesson:** Band-aid fixes to broken crypto are dangerous!

[a] https://appliedcryptography.page/paper/#mega-recovery

MEGA attacks: Summary

Note: Due to time constraints, we won't dive into the details of each attack

- **Attack 1 (RSA Key Recovery):** Exploit missing integrity protection to recover RSA private key in 512 logins[a]
- **Attack 2 (Plaintext Recovery):** Use recovered RSA key to build AES-ECB decryption oracle for all encrypted keys

- **Attack 3a (File Integrity - Oracle):** Forge files using decryption oracle to frame users with malicious content
- **Attack 3b (File Integrity - Key Reuse):** Forge files without oracle by exploiting MEGA's key obfuscation scheme
- **Attack 4 (RSA Decryption):** Bleichenbacher's attack on PKCS#1 v1.5 to decrypt RSA-encrypted keys

[a] Later improved to 2 logins: https://appliedcryptography.page/paper/#mega-recovery

MEGA attacks: Summary

Attack	Attempts	Prerequisites	Impact
RSA Key Recovery	2 logins	None	Share keys compromised
Plaintext Recovery	Varies	Attack 1	All keys compromised
Integrity Attack A	1	Attack 2	File forgery
Integrity Attack B	1	One AES block	File forgery
RSA Decryption	2^{17}	Weaker model	Chat/node keys

Root causes:

- No authenticated encryption (AES-ECB without MAC)
- Key reuse across different contexts
- Legacy cryptography (PKCS#1 v1.5)
- **Complex, ad-hoc design without formal analysis!**

Section 2

Attacks on MEGA and Nextcloud

Subsection 2.2

Attacks on Nextcloud

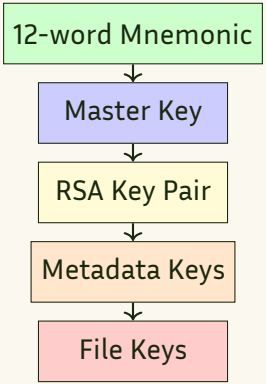
Nextcloud: Overview

- Open-source self-hosted cloud storage platform
- Founded in 2016 (fork of ownCloud)
- **Key statistics:**
  - 20+ million users (2017)
  - 400,000+ deployments (2022)
  - Used by German government, Amnesty International
- **E2EE claims:**
  - “Zero Knowledge” server
  - Secure against full server breach
  - Enterprise-grade encryption

Nextcloud’s logo.

Nextcloud: E2EE Design

- E2EE is **not enabled by default**
- Users must manually mark folders as E2EE
- **Encryption approach:**
  - Each file: AES-GCM with unique key
  - File keys encrypted by metadata key
  - Metadata keys encrypted by RSA-OAEP
- **Key rotation support:**
  - Array of metadata keys per folder
  - Uses highest-indexed key for encryption
  - Re-encrypts on each sync



Nextcloud: Key hierarchy

- **Mnemonic (top level):**
  - 12 random words
  - Generated by client
  - User must memorize/store
- **Master key:**
  - Derived from mnemonic
  - Encrypts RSA private key
- **RSA key pair:**
  - Public key stored in clear
  - Private key encrypted on server
  - Used for sharing folders
- **Metadata keys:**
  - One per E2EE folder
  - Encrypted with RSA-OAEP
  - Encrypts file keys and metadata
- **File keys (bottom level):**
  - Unique per file
  - Random 128-bit AES key
  - Encrypted by metadata key

Nextcloud vulnerabilities: Overview

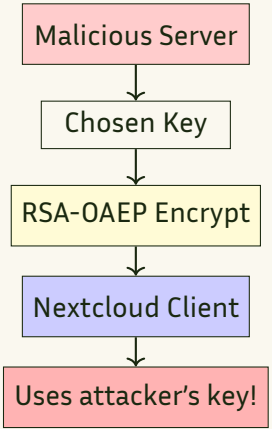
Three critical vulnerabilities found:[a]

1. **Key Insertion Attack**
   - Server can substitute metadata keys
   - Exploits lack of authentication in RSA-OAEP
2. **Ghost Key Attack**
   - Forces client to use all-zero encryption key
   - Exploits missing bounds checking
3. **IV Reuse in AES-GCM**
   - Same IV used when files are modified
   - Enables plaintext recovery and forgery

[a] https://appliedcryptography.page/paper/#nextcloud-break

Attack 1: Key Insertion Attack

- **Root cause:** RSA-OAEP provides confidentiality but **not authentication**
- **Attack scenario:**
  1. Server creates chosen metadata key
  2. Encrypts it with user's RSA public key
  3. Places ciphertext in user's folder
  4. Triggers key rotation
- **Impact:**
  - Server knows metadata key
  - Can decrypt all file keys
  - Can insert/modify files

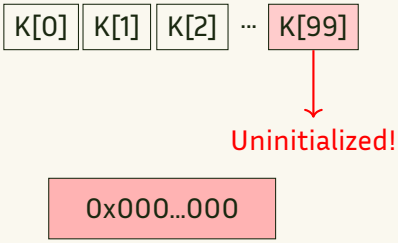


The authentication problem in public key encryption

- **What RSA-OAEP provides:**
  - Confidentiality: Only private key holder can decrypt
  - Chosen-ciphertext security (IND-CCA2)
- **What RSA-OAEP does NOT provide:**
  - Authentication: No proof of who created the ciphertext
  - Anyone with public key can encrypt
- **Nextcloud's assumption:**
  - "Only legitimate users will encrypt metadata keys"
  - But server has all public keys!
- **Solution approaches:**
  - Digital signatures on encrypted keys
  - Authenticated encryption (e.g., signcryption)
  - Key exchange protocols instead of PKE

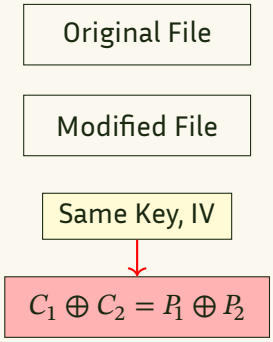
Attack 2: Ghost Key Attack

- **Vulnerability:** Missing bounds checking on metadata key array
- **Attack mechanism:**
  1. Server modifies key index
  2. Points to non-existent entry
  3. Client allocates default key
  4. Default is all zeros!
- **Result:**
  - Files encrypted with known key (`0x00...`)
  - Complete confidentiality loss
  - Trivial file forgery



Attack 3: IV Reuse in AES-GCM

- **Critical flaw:** Same IV used when file is modified
- **AES-GCM requirement:** Never reuse IV with same key
- **Consequences of IV reuse:**
  - Keystream repeats
  - XOR of plaintexts revealed
  - Authentication key recovery
- **Attack scenarios:**
  - **Plaintext recovery:** If attacker knows one version
  - **Forgery:** Create valid ciphertexts



Nextcloud's response

- **Initial patches:**
  - Added bounds checking (Ghost Key)
  - Generate new IV on file modification
  - But Key Insertion attack remained...
- **Drastic measure (2023):**
  - **Disabled E2EE file sharing entirely**
  - Could not fix authentication issues
  - Major feature regression
- **Lessons learned:**
  - Ad-hoc crypto designs are dangerous
  - Retrofitting security is difficult
  - Need formal analysis before deployment
  - Authentication is as important as confidentiality

Comparing MEGA and Nextcloud failures

MEGA

- No authenticated encryption
- Complex key hierarchy
- ECB mode usage
- Legacy crypto (PKCS#1 v1.5)
- Patches partially successful

Nextcloud

- Missing authentication
- Implementation bugs
- IV reuse in AES-GCM
- Bounds checking errors
- Had to disable features

Common theme: Complex, ad-hoc designs without formal security analysis!

Section 2

Attacks on MEGA and Nextcloud

Subsection 2.3

Additional Ecosystem Analysis

Additional attacks in the ecosystem (2024)

- Recent comprehensive analysis of 5 major E2EE providers[a]
- Collectively serving 22+ million users
- **Key finding:** Widespread cryptographic failures across ecosystem
- **Common patterns:**
  - Unauthenticated key material (Sync, pCloud)
  - Unauthenticated encryption modes (Icedrive, Seafile)
  - Missing integrity protection
  - Protocol downgrade attacks
- **Impact:** Most “secure” providers fail basic security requirements

[a] https://appliedcryptography.page/paper/#cloud-broken

Attack highlights: Breaking confidentiality

- **Key replacement attacks**
  - Sync/pCloud: No authentication of public key encrypted material
  - Server replaces user keys with attacker-controlled keys
  - All future uploads compromised
- **Protocol downgrade (Seafile)**
  - Forces client to use weak KDF
  - 3 iterations of SHA1 instead of PBKDF2
  - Enables offline password cracking
- **Link sharing leakage (Sync)**
  - Password embedded in URL path
  - Sent to server on every click
  - Complete confidentiality breach
- **Unauthenticated chunking**
  - Seafile/pCloud: Can reorder/remove file chunks
  - Icedrive: CBC mode allows controlled tampering
  - Files corrupted without detection

The state of E2EE cloud storage: A broken ecosystem

- **Systemic failures across providers:**
  - 4 out of 5 analyzed providers have critical vulnerabilities
  - Only Tresorit showed reasonable security design
  - Common mistakes repeated independently
- **Root causes echo MEGA/Nextcloud:**
  - Ad-hoc cryptographic designs
  - Misunderstanding primitive guarantees
  - No formal security analysis
  - Missing authentication at multiple levels
- **Key lesson:** E2EE cloud storage is not a “solved problem”
- **Need for:** Standardized, formally analyzed protocols

Section 3

Authentication: Moving Beyond Passwords

Beyond password hashing: Zero-knowledge protocols

- Password hashing has fundamental limitations:
  - Server must receive password (or hash) to verify
  - Vulnerable to offline dictionary attacks
  - Trade-off between security and performance
- **Zero-knowledge password protocols** offer better security:
  - Server never sees password or password-equivalent
  - No offline dictionary attacks possible
  - Cryptographic proof of password knowledge

SRP (Secure Remote Password)

- Developed by Tom Wu at Stanford (1998)
- **Key properties:**
  - Password never transmitted, even encrypted
  - Mutual authentication between client and server
  - Generates session key for subsequent encryption
  - Patent-free and standardized (RFC 2945)
- **How it works:**
  - Registration: Client sends verifier derived from password
  - Authentication: Zero-knowledge proof using verifier
  - Based on discrete logarithm problem
- **Adoption:** Used by Apple iCloud, 1Password, ProtonMail

OPAQUE: Next-generation password authentication

- Developed by Jarecki, Krawczyk, and Xu (2018)
- **Advantages over SRP:**
  - Stronger security proofs in UC framework
  - Protection against pre-computation attacks
- **Key innovation:** Oblivious PRF (OPRF)
  - Server helps compute PRF without learning input
  - Prevents server from testing password guesses
- Being standardized by IETF (OPRF component: RFC 9497)
- Early adoption by WhatsApp for backup encryption

Forward secrecy in OPAQUE

- **What is forward secrecy for passwords?**
  - If server is compromised *after* authentication
  - Attacker cannot test password guesses offline
  - Past sessions remain secure even with server data
- **Contrast with traditional password hashing:**
  - Hash database breach → offline cracking possible
  - OPAQUE breach → offline cracking possible, but no pre-computation advantage
  - Past session keys remain secure (AKE forward secrecy)

Comparing authentication approaches

Password Hashing

- ✓ Simple to implement
- ✓ Well understood
- ✗ Offline attacks
- ✗ Server sees password

SRP

- ✓ No eavesdropper offline attacks
- ✓ Proven secure
- ~ Complex protocol
- ✗ Pre-computation attacks

OPAQUE

- ✓ Strongest security
- ✓ Forward secrecy
- ✗ New protocol
- ✗ Limited deployment

For E2EE storage: Can derive encryption keys from authentication protocol outputs

OPAQUE: How it works under the hood

- **OPAQUE combines three key components:**
  - **OPRF (Oblivious PRF):** Server helps compute function without seeing input
  - **Authenticated Key Exchange:** Derives session keys after password verification
  - **Password hardening:** Makes brute-force attacks computationally expensive
- **The magic:** Server stores a “password file” but:
  - File doesn’t contain password or hash
  - Server can’t test password guesses offline
  - Client proves password knowledge without revealing it
    - You could even say they perform a *zero knowledge proof*!

What is an OPRF?

Oblivious Pseudorandom Function

- **Definition:** A two-party protocol where:
  - Client has input x
  - Server has key k
  - Client learns $F(k, x)$ and nothing else
  - Server learns nothing about x
    - You could even say they perform a *multi-party computation*!
- **Key properties:**
  - **Oblivious:** Server doesn't learn client's input
  - **Verifiable (VOPRF variant):** Client can verify correct computation
  - **Deterministic:** Same input always gives same output

OPAQUE Registration: Step by step

Client actions:

1. Choose password pw
2. Run OPRF with server:[a]
   - Send blinded $H(pw)^r$
   - Get back $(H(pw)^r)^k$
   - Unblind:
     $rwd = ((H(pw)^r)^k)^{1/r} = H(pw)^k$
3. Generate random private key sk_c
4. Encrypt: $env = \text{Enc}(rwd, sk_c)$
5. Send (env, pk_c) to server

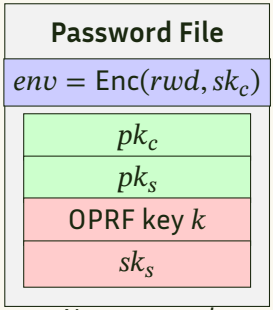
Server actions:

1. Generate OPRF key k
2. Respond to OPRF:
   - Receive blinded value
   - Return $(blinded)^k$
3. Generate key pair (sk_s, pk_s)
4. Store password file:
   - Client's env (encrypted)
   - Client's pk_c
   - Own sk_s, pk_s
   - OPRF key k

[a] Here, our chosen OPRF operates in a cyclic group $\mathbb{G}$ of prime order q. The blinding factor r is sampled uniformly at random from $\mathbb{Z}_q$, and k is the server's OPRF key. I just made this OPRF up for the purposes of this slide and I don't know if it's secure (it probably is?)

What's in the password file?

- **Client's envelope (env):**
  - Encrypted with $rwd = H(pw)^k$
  - Contains client's private key sk_c
  - Server can't decrypt without password
- **Public keys:**
  - Client's pk_c (for key exchange)
  - Server's pk_s (for authentication)
- **Server's secrets:**
  - OPRF key k (for password evaluation)
  - Server private key sk_s



No password or hash stored!

OPAQUE login (after registration)

1. **Client starts OPRF:**
   - Blinds password: $\alpha = H(pw)^r$
   - Sends α to server
   - Note: This r is freshly randomized - different from registration and each login
2. **Server responds:**
   - Computes $\beta = \alpha^k = H(pw)^{rk}$
   - Sends back β and encrypted envelope env
3. **Client recovers private key:**
   - Unblinds: $rwd = \beta^{1/r} = H(pw)^k$
   - Decrypts: $sk_c = \text{Dec}(rwd, env)$
   - If wrong password, decryption fails!
4. **Authenticated key exchange:**
   - Client and server run AKE using their key pairs
   - Derives session key and export key
   - Mutual authentication achieved

Why can't the server test passwords?

What the server sees:

- Random group element α
- No relation to password visible
- Different α each login (random r)
- Can't extract $H(pw)$ from α

What prevents offline attacks:

- Server doesn't know $H(pw)$
- Can't verify guesses without client
- Each guess requires online interaction
- Rate limiting becomes enforceable

Key insight: The OPRF makes the server "blind" to the password while still enabling authentication

OPAQUE's security guarantees

Against server compromise:

- ✓ No pre-computation advantage
- ✓ Forward secrecy for session keys
- ✓ Past sessions remain secure

Against active attacks:

- ✓ Mutual authentication
- ✓ Key confirmation
- ✓ Protection from phishing

Bonus: Provides both authentication *and* key derivation - perfect for E2EE storage!

Section 4

Case Study: WhatsApp End-to-End Encrypted Backups

Section 4

Case Study: WhatsApp End-to-End Encrypted Backups

Subsection 4.1

WhatsApp’s Design

WhatsApp End-to-End Encrypted Backups: Overview

- WhatsApp messages have been E2EE since 2016
- **Problem:** Backups in iCloud/Google Drive were not E2EE
- **Solution (2021):** End-to-end encrypted backups
- **Key innovation:** HSM-based Backup Key Vault
  - Hardware Security Module for key storage
  - Password-protected key recovery
  - Brute-force protection
- **User choice:**
  - Password-based recovery (via HSM)
  - 64-digit key (user manages)

WhatsApp logo.

The backup encryption challenge

- **User expectations:**
  - Recover chats after losing phone
  - Transfer chats to new device
  - No complex key management
- **Security requirements:**
  - WhatsApp cannot read backups
  - Cloud providers cannot read backups
  - Protection against brute-force attacks
- **Traditional approach:** Trust cloud provider
- **WhatsApp's approach:** Client-side encryption with secure key recovery

Hardware Security Modules (HSMs)

What are they?

- **Definition:** Dedicated cryptographic processors designed for key protection
- **Key features:**
  - Hardware-based key storage and generation
  - Tamper-resistant physical security
  - Cryptographic operations in secure boundary
  - FIPS 140-2 Level 3/4 certified
- **Security properties:**
  - Keys never exist in plaintext outside HSM
  - Physical intrusion triggers key deletion
  - Rate limiting and access controls
  - Audit logging of all operations
- **Common uses:**
  - PKI root certificate authorities
  - Payment card industry (PIN verification)
  - Database encryption key management
  - Code signing for software integrity

Signal's SVR3: Multi-enclave key recovery

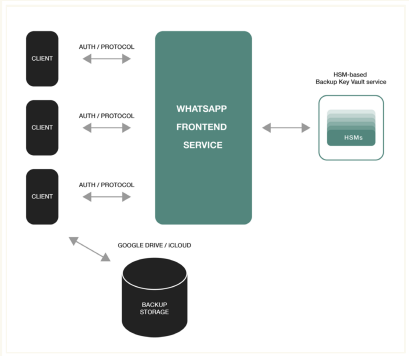
Quick aside

- Signal faced same challenge: E2EE backups need secure key recovery
- **SVR3 (Secure Value Recovery 3)**[a]**:** PIN-based key recovery
  - Distributes trust across 3 different hardware enclaves
  - Intel SGX (Azure), AMD SEV-SNP (Google Cloud), Nitro (AWS)
  - Attacker must compromise ALL three to access keys
- **Key features:**
  - Uses Password Protected Secret Sharing (PPSS)
  - Limits PIN guesses via secure hardware
  - No single point of failure
  - Cost: $0.0025/user/year
- Already deployed to millions of Signal users

[a] https://appliedcryptography.page/paper/#signal-recovery

WhatsApp Encrypted Backups: System architecture

- **Client side:**
  - $K \twoheadleftarrow \{0,1\}^{256}$
  - Encrypts backup with K
  - Uploads to iCloud/Google Drive
- **WhatsApp infrastructure:**
  - Front-end service (ChatD)
  - HSM-based Backup Key Vault
  - Geographically distributed
- **Security properties:**
  - Cloud provider sees only ciphertext
  - K never leaves client unencrypted
  - Password never known to WhatsApp



Source: https://appliedcryptography.page/paper/#whatsapp-backups

HSM-based Backup Key Vault

The safe deposit box analogy

Traditional Bank Safe Deposit Box

- User has unique key
- Bank cannot open box
- Requires physical presence
- Lost key = lost contents

WhatsApp's Key Vault

- Password protects key
- WhatsApp cannot access
- Remote recovery possible
- Password attempts limited

Now the problem becomes: how to store encrypted K?

- WhatsApp uses OPAQUE for password-based key storage
- **Key properties of OPAQUE:**
  - Server never learns password
  - No offline dictionary attacks
  - Derives encryption key from password
  - Strong security proofs

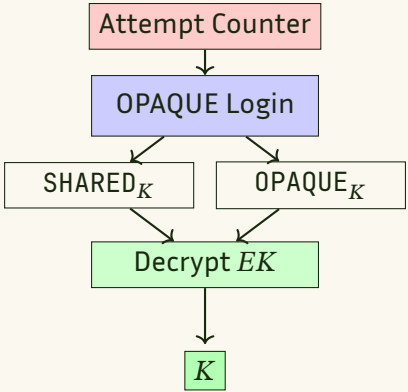
- **How it works:**
  - Registration: Client creates password-protected record
  - Retrieval: Client proves password knowledge
  - Server helps compute key without learning password
- **Advantage over simple password hashing:**
  - HSM enforces rate limiting
  - No password-equivalent stored

Registration process

1. User enters password
2. Client starts OPAQUE registration
3. Server returns registration response + signature
4. Client completes registration:
   - Derives $\texttt{OPAQUE}_K$
   - Sends $\text{AES-GCM}(\texttt{OPAQUE}_K, K)$ to server
5. Server stores encrypted record

Key retrieval process

1. Client initiates OPAQUE login with password
2. Server decrements attempt counter
   - Enforced via HSM
3. Server returns credential response
4. Client completes login:
   - Derives $\texttt{SHARED}_K$ and $\texttt{OPAQUE}_K$
   - Sends finalization message
5. Server returns encrypted key (EK)
6. Client decrypts:
   - First with $\texttt{SHARED}_K$
   - Then with $\texttt{OPAQUE}_K$
   - Recovers backup key K



Security features

Brute-force protection

- HSM enforces attempt limits
- After threshold: Key permanently inaccessible
- Rate limiting in hardware

Infrastructure resilience

- 5-datacenter deployment
- Majority consensus system
- Tolerates multiple failures
- Geographic distribution

Transport security: Noise Protocol Framework for client-server communication (here used as an alternative to TLS, but in reality it's a whole thing that lets you design a wide variety of protocols)

Alternative: User-managed 64-digit key

- **For users who prefer full control:**
  - 64-digit encryption key option
  - Key never sent to WhatsApp
  - User must store key securely
- **Trade-offs:**
  - ✓ Maximum security - no trust required
  - ✓ No brute-force risk
  - ✗ User responsibility for key management
  - ✗ Lost key = lost backups forever
- **Recommendation:** Only for technically sophisticated users

WhatsApp's design principles

- **Use proven protocols:** OPAQUE for password authentication
- **Hardware security:** HSMs for tamper-resistant key storage
- **Defense in depth:**
  - Client-side encryption
  - Zero-knowledge password protocol
  - Rate limiting in hardware
  - Geographic distribution
- **User choice:** Password vs self-managed key
- **Clear threat model:**
  - Protects against cloud provider access
  - Protects against WhatsApp compromise
  - Limits brute-force attacks

Lessons from WhatsApp's approach

What they did right:

- Used established OPAQUE protocol
- Hardware-backed security
- Simple, clear design
- Published detailed whitepaper
- Gradual rollout with testing

Key takeaways:

- Don't reinvent authentication
- User education is crucial
- Transparency builds trust
- Complexity should be hidden

Compare with MEGA/Nextcloud: Standard protocols vs. custom designs

Section 4

Case Study: WhatsApp End-to-End Encrypted Backups

Subsection 4.2

Critiquing WhatsApp's Design

Specification ambiguity: A major flaw

- **The problem:** WhatsApp's specification is ambiguous about $\texttt{OPAQUE}_K$
- **Two possible interpretations:**
  - **Option 1:** OPAQUE's export key (client-only secret)
  - **Option 2:** OPAQUE's shared session key (known to both parties)
- **Why this matters:**
  - If Option 2: Server learns $\texttt{OPAQUE}_K$
  - Server could decrypt backup key directly!
  - Completely breaks the security model
- **This type of ambiguity is dangerous:**
  - Different implementations may choose differently
  - Security analysis becomes impossible
  - Opens door to implementation vulnerabilities
- **Lesson:** Cryptographic specifications must be *unambiguous*

Critical limitation: Trust in WhatsApp's infrastructure

- **The fundamental trust problem:**
  - WhatsApp claims to use HSMs, but users cannot verify
  - HSMs are owned and operated by WhatsApp
  - No independent audit or verification possible
  - Could be software emulation instead
- **What users must trust:**
  - WhatsApp actually deployed HSMs as described
  - HSMs are configured correctly
  - Rate limiting is enforced as claimed
  - WhatsApp doesn't have backdoor access

The HSM security theater problem

What WhatsApp claims:

- Hardware-enforced rate limiting
- Tamper-resistant key storage
- Geographic distribution
- No backdoor access

What users can verify:

- Nothing!
- No attestation provided
- No third-party audits
- Closed-source implementation

Potential vulnerabilities in WhatsApp's design

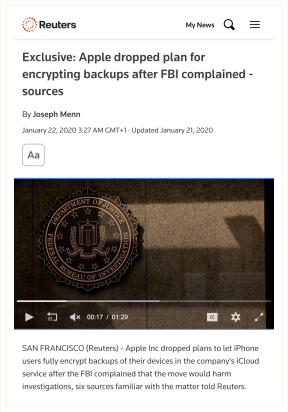
- **Silent key escrow:**
  - HSM could have backdoor for data extraction
  - HSM could literally be one guy in a basement memorizing everything
  - No way for users to detect this
- **Selective targeting:**
  - Different behavior for specific accounts
  - Disable rate limiting for law enforcement
  - Return fake "attempt exceeded" to user
- **Implementation substitution:**
  - Replace HSM with software for some users
  - Use weaker entropy for key generation
  - Log passwords before OPAQUE processing

The attestation problem

- **What's missing:**
  - Remote attestation from HSMs
  - Cryptographic proof of HSM usage
  - Public audit logs
  - Third-party verification
- **Technical solutions exist:**
  - HSMs can provide attestation
  - Intel SGX, AMD SEV offer remote attestation
  - Blockchain for public audit logs

- **Why not implemented?**
  - Complexity for users?
  - Business reasons?
  - Legal requirements?
  - Law enforcement pressure?

Law enforcement pressure?



Reuters My News

Exclusive: Apple dropped plan for encrypting backups after FBI complained - sources

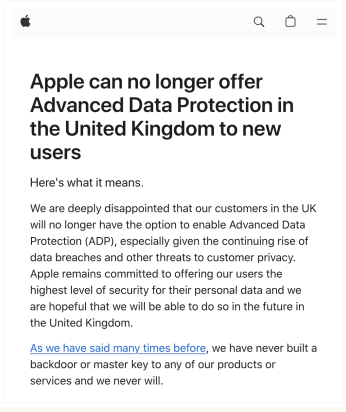
By Joseph Menn

January 22, 2020 3:27 AM GMT+1 · Updated January 21, 2020

Aa

SAN FRANCISCO (Reuters) - Apple Inc dropped plans to let iPhone users fully encrypt backups of their devices in the company's iCloud service after the FBI complained that the move would harm investigations, six sources familiar with the matter told Reuters.

Source: Reuters



Apple can no longer offer Advanced Data Protection in the United Kingdom to new users

Here's what it means.

We are deeply disappointed that our customers in the UK will no longer have the option to enable Advanced Data Protection (ADP), especially given the continuing rise of data breaches and other threats to customer privacy. Apple remains committed to offering our users the highest level of security for their personal data and we are hopeful that we will be able to do so in the future in the United Kingdom.

As we have said many times before, we have never built a backdoor or master key to any of our products or services and we never will.

Source:
`https://support.apple.com/en-us/122234`

The 64-digit key mystery

- **Why 64 digits?**
  - 64 digits = ∼212 bits of entropy
  - Much more than needed for 128-bit AES key
  - Difficult for humans to manage
  - Error-prone to transcribe
- **We already have better solutions:**
  - BIP39 mnemonic phrases (12-24 words)[a]
  - Used successfully by cryptocurrency wallets
  - Easier to write down and verify
  - Built-in error detection

- **Possible explanations:**
  - Deliberate friction to push HSM option?
  - Making self-custody intentionally hard?
  - Or maybe using passphrases is just hard to localize into many languages

[a] Argument against: hard to localize into many languages

A more honest assessment of WhatsApp backups

- **What WhatsApp backups actually provide:**
  - Protection from cloud providers (Google/Apple)
  - Some protection from external hackers
  - Convenient key recovery
- **What they DON'T provide:**
  - Full protection from WhatsApp itself
  - Fully verifiable security guarantees
- **Alternatives:**
  - Threshold cryptography across providers
  - Client-side key derivation only
  - Transparent, auditable systems

Section 5

Designing an End-to-End Cloud Storage Protocol

Section 5

Designing an End-to-End Cloud Storage Protocol

Subsection 5.1

Lessons Learned

Lessons from MEGA and Nextcloud failures

What went wrong?

- **Ad-hoc design:** Both created custom cryptographic protocols
- **No formal analysis:** Security claims without proofs
- **Complexity spiral:** Features added without security review
- **Missing fundamentals:**
  - MEGA: No authenticated encryption
  - Nextcloud: No sender authentication
  - Both: Improper use of cryptographic primitives
- **Patch difficulties:**
  - MEGA: Multiple rounds of incomplete fixes
  - Nextcloud: Had to disable core features

The danger of ad-hoc designs

- **Common mistakes in ad-hoc designs:**
  - Misunderstanding primitive guarantees
  - Ignoring composition problems
  - No clear threat model
  - Feature creep without security analysis
- **Why it happens:**
  - Overconfidence in understanding
  - Time/resource constraints
  - Lack of cryptographic expertise
  - “It encrypts, so it’s secure”

Principles of secure protocol design

1. **Start with a clear threat model**
   - What are you protecting against?
   - What can the adversary do?
   - What are acceptable trade-offs?
2. **Use established cryptographic patterns**
   - Authenticated encryption (AES-GCM, ChaCha20-Poly1305)
   - Standard key exchange (X3DH, Noise Protocol Framework)
   - Proven constructions (don't invent new ones!)
3. **Keep it simple**
   - Complexity is the enemy of security
   - Each feature multiplies attack surface
   - Can you explain your protocol in one page?

Reminder: formal analysis

- **What is formal analysis?**[a]
  - Automated using software
  - Mathematical proofs of security
  - Clear security definitions
  - Reduction to hard problems
  - Machine-verified proofs
- **Benefits:**
  - Catches subtle flaws
  - Provides confidence
  - Clear security guarantees
  - Guides implementation

- **Types of analysis:**
  - **Computational:** Reduction to hard problems
  - **Symbolic:** Protocol logic verification
  - **UC Framework:** Composable security
  - **Game-based:** Step-by-step proofs
- **Tools available:**
  - ProVerif, Tamarin
  - EasyCrypt, CryptoVerif
  - F*

[a] Coming soon in our upcoming topic sessions on High-Assurance Cryptography!

Security definitions matter

MEGA's mistake: Assumed encryption provides integrity

- Used AES-ECB without MAC
- Encryption ≠ Authenticated encryption
- Led to complete key recovery

Nextcloud's mistake: Assumed encryption provides authentication

- RSA-OAEP only provides confidentiality
- Anyone can create valid ciphertexts
- Led to key insertion attacks

Lesson: Understand exactly what each primitive guarantees!

Protocol design methodology

- **Requirements gathering**
  - Security goals (confidentiality, integrity, authentication)
  - Functionality requirements
  - Performance constraints
- **Threat modeling**
  - Define adversary capabilities
  - Identify trust assumptions
  - Document acceptable risks
- **Protocol design**
  - Use standard building blocks
  - Create formal specification
  - Keep complexity minimal
- **Security analysis**
  - Formal proofs or verification
  - External review
  - Test edge cases

Learning from other successful protocols

- **Signal Protocol**
  - Formal security proofs
  - Clear threat model
  - Simple, modular design
  - Widely analyzed
- **WireGuard**
  - Minimal codebase
  - Fixed crypto choices
  - Formal verification
  - Clear specification
- **TLS 1.3**
  - Removed legacy features
  - Formal analysis during design
  - Simplified handshake
  - Clear security properties
- **Common themes:**
  - Simplicity over features
  - Formal analysis early
  - Clear documentation
  - Conservative choices

Key takeaways for protocol designers

- **Don’t roll your own crypto**
  - Use established, analyzed protocols
  - If you must innovate, get expert review
- **Security is not just encryption**
  - Authentication, integrity equally important
  - Consider key management, revocation
- **Formal analysis is worth it**
  - Catches issues before deployment
  - Provides clear security guarantees
- **Simplicity is security**
  - Every feature adds complexity
  - Complex protocols have more bugs
- **Learn from failures**
  - Study what went wrong
  - Apply lessons to new designs

Section 5

Designing an End-to-End Cloud Storage Protocol

Subsection 5.2

A Formal Treatment of End-to-End Encrypted Cloud Storage

Next: A formal treatment from the literature

What follows is based on:

"A Formal Treatment of End-to-End Encrypted Cloud Storage"

https://appliedcryptography.page/paper/#cloud-storage

by Matilda Backendal, Hannah Davis, Felix Günther, Miro Haller, and Kenneth G. Paterson

- **Why this paper?**
  - First comprehensive formal treatment of E2EE cloud storage
  - Addresses the exact failures we've seen in MEGA/Nextcloud
  - Shows how to do protocol design *right*
- **Key contributions:**
  - Formal security definitions for E2EE storage
  - Clean, analyzable protocol design
  - Game-based security proofs

A formal treatment: Building blocks

Core cryptographic primitives:

- **PRF (Pseudorandom Function):** $F : \{0,1\}^\kappa \times \{0,1\}^* \to \{0,1\}^\kappa$
- **MAC (Message Authentication Code):** For integrity verification
- **AEAD (Authenticated Encryption):** AES-GCM or ChaCha20-Poly1305
- **2HashDH OPRF:** Password hardening against offline attacks

Key insight: Use standard, proven primitives - don't invent new ones!

The 2HashDH OPRF: Motivation

- **Problem:** How to derive keys from passwords without server learning them?
- **Challenge:** Server needs to help with computation, but shouldn't learn password
- **Solution:** Oblivious Pseudorandom Function (OPRF) (also used by OPAQUE)
  - Client has secret input (account ID + password)
  - Server has key k
  - Together compute $F(k, \text{input})$ without server learning input
- **In our protocol:**
  - Client generates OPRF key during registration
  - Sends key to server (ensures honest key generation)
  - Later uses OPRF to derive keys from password

The 2HashDH OPRF: How it works

Setup: Group $\mathbb{G}$ of prime order q, hash functions H_1 and H_2[a]

- **Goal:** Client computes $H_2(x, H_1(x)^k)$ without revealing x to server
- **Protocol steps:**
  1. Client picks random blinding factor $r \twoheadleftarrow \mathbb{Z}_q$
  2. Client sends blinded value: $\alpha = H_1(x)^r$ to server
     - Client hashes their secret input x to get $H_1(x)$, raises it to the power of r, and sends this "blinded" result to the server - the server cannot determine x from this
  3. Server computes: $\beta = \alpha^k = H_1(x)^{rk}$ and returns it
  4. Client unblinds: $\beta^{1/r} = H_1(x)^k$
     - The client removes the blinding by raising the server's response to the power of $1/r$ (the multiplicative inverse of r), which cancels out the r in the exponent, leaving just $H_1(x)^k$
  5. Client outputs: $y = H_2(x, \beta^{1/r})$

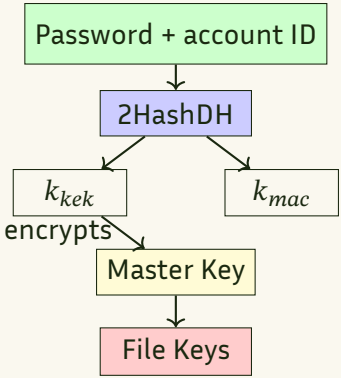
[a] Ideally memory-hard and slow

The 2HashDH OPRF: How it works

- **Security properties:**
  - Server sees only random group element α (learns nothing about x)
  - Client gets consistent PRF output
  - Memory-hard hash functions prevent brute-force attacks

Scheme architecture: Clean key hierarchy

- **Password-derived keys:**
  - Raw secret rw from OPRF
  - Key encryption key k_{kek}
  - MAC key k_{mac}
- **Master key (k_{mk}):**
  - Randomly generated
  - Encrypted with k_{kek}
  - Enables password changes!
- **File keys:**
  - Unique per file
  - Encrypted with master key
  - Shared via headers



Registration and authentication flow

Registration:

1. Client samples OPRF key k, has account ID (aid)
2. Computes $rw = \text{OPRF}(aid, pw)$
3. Derives k_{kek}, k_{mac} from rw
4. Generates random master key
5. Encrypts master key with k_{kek}
6. Sends to server: aid, k, k_{mac}, encrypted master key

Authentication:

1. Client initiates OPRF with server
2. Server sends session ID (sid)
3. Client recovers rw, derives keys
4. Client MACs transcript with k_{mac}
5. Server verifies MAC
6. Server returns encrypted master key
7. Client decrypts master key

File operations: Simple and secure

Upload (put):

- Generate random file key
- AEAD-encrypt file with file key
- AEAD-encrypt file key with master key
- Store header (encrypted file key)
- Store encrypted file content

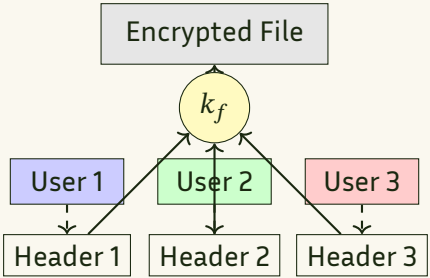
Download (get):

- Fetch encrypted header
- Decrypt file key with master key
- Fetch encrypted file
- Decrypt file with file key
- All bound to file ID via AEAD

Separating headers from content enables efficient sharing!

Sharing design: Each owner has their own header

- **Sharing process:**
  1. Owner fetches file key
  2. Shares key out-of-band
  3. Server adds new owner
  4. Recipient wraps key under their master key
- **Benefits:**
  - No re-encryption on updates
  - Independent key management
  - Efficient for large files
  - Password changes don't affect shares



Same file key,
different headers

Security properties achieved

Against malicious server:

- ✓ No offline password attacks (OPRF)
- ✓ Authenticated encryption throughout
- ✓ Unique account ID prevents correlation

Design advantages:

- ✓ Password changes without re-encryption
- ✓ Efficient file sharing
- ✓ Simple, analyzable design
- ✓ Uses only standard primitives

This scheme has formal security proofs!

Comparing with MEGA/Nextcloud

MEGA

- ✗ Custom crypto
- ✗ No AEAD
- ✗ Complex design
- ✗ No formal analysis

Nextcloud

- ✗ Missing authentication
- ✗ Implementation bugs
- ✗ Ad-hoc design
- ✗ Feature creep

This scheme

- ✓ Standard primitives
- ✓ AEAD everywhere
- ✓ Clean hierarchy
- ✓ Formal proofs

Simplicity and formal analysis make the difference!

Key takeaways

- **End-to-end encryption is hard to get right**
  - Even major companies make critical mistakes
  - Ad-hoc designs inevitably fail
  - Retrofitting security doesn't work
- **Use established cryptographic building blocks**
  - Don't roll your own crypto
  - Understand what primitives actually guarantee
  - Simplicity beats cleverness
- **Formal analysis is essential**
  - Catches subtle vulnerabilities
  - Provides clear security guarantees
  - Should be done *before* deployment
- **Trust models matter**
  - Be honest about what you're protecting against
  - Users need verifiable security, not promises
  - Transparency builds trust

And remember...

The best cryptographic protocol
is one that's
simple enough to understand
and *proven to be secure*

"Complexity is the enemy of security"

Symbolic software

Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.4: End-to-End Encrypted Cloud Storage

Nadim Kobeissi
https://appliedcryptography.page